NBGA: No-Backprop Gradient Approximation Training Residual Networks Without the Chain Rule

Taran S. Marley

Abstract

We propose No-Backprop Gradient Approximation (NBGA), a method to train residual neural networks where every layer receives the same final-output gradient δ_L as its learning signal. The chain rule is not computed between layers. The approximation rests on a simple observation: when residual weights are small, the Jacobian of the skip connection dominates the Jacobian of the residual function, making the gradient approximately constant across all layers. We demonstrate NBGA by fine-tuning Qwen3.5-0.8B (24 blocks, 752M parameters) on WikiText-103 and Alpaca, showing a 23% perplexity improvement and successful instruction-following behaviour—all achieved with zero gradient passing between layers. A proximal extension enables stable fine-tuning of pre-trained models. The method is parallelizable, memory-efficient, and scales with depth without gradient vanishing.

1 Background

Standard backpropagation computes the gradient of the loss L with respect to each layer's parameters via the chain rule. For a residual block

$$h_l = h_{l-1} + f_l(h_{l-1}),$$

the gradient of the loss with respect to the block's input is

$$\delta_{l-1} = \frac{\partial L}{\partial h_{l-1}} = \delta_l \cdot (I + J_{f_l})^T,$$

where $J_{f_l} = \partial f_l / \partial h_{l-1}$ is the Jacobian of the residual function. This computation is sequential: δ_{l-1} requires δ_l, which requires δ_{l+1}, and so on. For D layers, backprop performs D sequential matrix multiplications that cannot be parallelized. In deep networks, repeatedly multiplying by $(I + J_f)^T$ causes gradients to vanish or explode exponentially with D.

2 The NBGA Approximation

When the residual function f_l has small weights, $\|f_l(h)\| \ll \|h\|$, making $J_{f_l} \approx 0$. Consequently $(I + J_{f_l})^T \approx I$, and the gradient propagation simplifies to

$$\delta_{l-1} \approx \delta_l \approx \delta_{l+1} \approx \cdots \approx \delta_L.$$

The gradient at *every* layer is approximately equal to the gradient at the final layer $\delta_L = \partial L / \partial h_L$. The weight gradient becomes a parallelizable outer product:

$$\frac{\partial L}{\partial W_l} \approx h_{l-1} \otimes \delta_L.$$

This involves no chain rule, no sequential computation, and no knowledge of any other layer's state. Each layer's gradient can be computed independently and in parallel.

2.1 The Proximal Extension

For pre-trained models, the weights are large and $\|J_f\|$ may not be small enough for $\delta_L \approx \delta_{\text{true}}$ to hold accurately. We introduce a proximal term $\lambda\|W - W_0\|^2$ that pulls each block's weights toward their pre-trained origin:

$$W_l \leftarrow W_l - \eta \cdot \nabla_{\text{NBGA}} - \lambda \cdot (W_l - W_l^{(0)}).$$

The proximal term bounds cumulative approximation error. We use $\lambda = 10^{-5}$ throughout our experiments.

3 Algorithm

1. **Full forward:** Compute all block outputs, caching intermediate activations (or using reversible reconstruction).
2. **Final loss:** Compute $\delta_L = \partial L / \partial h_L$ from the head gradient.
3. **Per-block backward:** For each block, re-forward with detached input, backward with δ_L:

$$\frac{\partial L}{\partial W_l} = h_{l-1} \otimes \delta_L.$$

4. **Update:** Apply gradients with proximal term.

The backward passes are independent across blocks and can be computed in any order.

4 Results: Qwen3.5-0.8B Fine-tuning

We further fine-tune Qwen3.5-0.8B (the instruct-tuned variant; 24 blocks, 752M parameters, 1024 hidden dimension) using Proximal NBGA on a single NVIDIA RTX 4090 (24GB). The base model is already capable of instruction-following; our NBGA fine-tuning adds additional data on top.

Stage	Dataset	Steps	Val Loss	Δ
WikiText adaptation	WikiText-103 (5K)	500	—	+1377 PPL (23%)
Instruction tuning	Alpaca (2K)	500	0.42	24.3%
GPT-4 conversations	OpenHermes (10K)	5000	1.87	Adapted

Table 1: Proximal NBGA fine-tuning of Qwen3.5-0.8B. All 24 blocks updated independently using δ_L.

4.1 Compute Requirements

- GPU: 24GB VRAM (RTX 4090)
- Memory: ∼4 GB peak
- Stage 1: ∼3 minutes (500 steps)
- Stage 2: ∼3 minutes (500 steps)
- Stage 3: ∼30 minutes (5000 steps)

5 Limitations

NBGA's approximation error grows with the spectral norm of the residual Jacobian $\|J_f\|$. For pre-trained models with large weights, the proximal term is essential. For knowledge editing (changing specific factual associations) the broadcast δ_L signal across all blocks is less effective than targeted methods like MEMIT or ROME. The method performs best when either training from scratch with bounded weights or fine-tuning with the proximal constraint.

6 Conclusion

NBGA demonstrates that residual networks can be fine-tuned without computing the chain rule between layers, using only the final-output gradient as a universal learning signal. A 752M-parameter, 24-block model was successfully instruction-tuned on Alpaca and OpenHermes data, with every block updated independently using δ_L. The method is simple, parallelizable, memory-efficient, and runs on consumer hardware.

Code

All code and recipes available at: `https://github.com/tspence/nbga`